

LearnHire

Buku Pedoman Teknis & Bedah Struktur Berkas

Eksplorasi Mendalam Setiap Direktori, Berkas Sumber Daya, Model, Controller, Rute, Antarmuka, dan Skema Basis Data

INFORMASI AKADEMIK & SPESIFIKASI PROYEK

Mata Kuliah	: Desain dan Rekayasa Sistem (DRS) B
Departemen	: Teknik Komputer
Fakultas	: Teknologi Elektro dan Informatika Cerdas (FTEIC)
Institusi	: Institut Teknologi Sepuluh Nopember (ITS)
Dosen Pengampu	: Dr. Supeno Mardi Susiki Nugroho, S.T., M.T.
Arsitektur Perangkat Lunak	: Model-View-Controller (MVC) & RESTful API Architecture
Lingkungan Backend	: Node.js (v20.18.0 LTS), Express.js (v4.18)
Basis Data Relasional	: MySQL / MariaDB (XAMPP Server Port 3306)
Teknologi Frontend	: HTML5, CSS3 Glassmorphism, Vanilla JavaScript ES6+
Sistem Keamanan	: JWT Bearer Token, Bcryptjs Salt Hashing, Helmet CSP, CORS
Cakupan Dokumentasi	: Seluruh 65 File Aktif Tanpa Menggunakan Mock Data

Contents

1	Pengantar & Peta Arsitektur Tiga Tingkat (Three-Tier)	2
1.1	Alur Interaksi Antar Layer	2
2	Bedah Berkas Root & Direktori Database	3
2.1	Berkas pada Root Direktori (learnhire/)	3
2.2	Direktori Basis Data (database/)	3
3	Bedah Direktori Backend (backend/)	4
3.1	Berkas Konfigurasi & Entry Point Root Backend	4
3.2	Subdirektori Konfigurasi (backend/config/)	5
3.3	Subdirektori Dokumentasi API (backend/docs/)	5
3.4	Subdirektori Middleware (backend/middleware/)	5
3.5	Subdirektori Model Data (backend/models/)	7
3.6	Subdirektori Controller Backend (backend/controllers/)	9
3.7	Subdirektori Routing Backend (backend/routes/)	12
4	Bedah Direktori Frontend (frontend/)	13
4.1	Landing Page Publik & Komponen Bersama	13
4.2	Pusat Gaya Antarmuka (frontend/assets/css/)	13
4.3	Pustaka JavaScript Klien Bersama (frontend/assets/js/)	15
4.4	Halaman Web Modular (frontend/pages/)	16
4.5	Skrip Klien Per Halaman (frontend/assets/js/pages/)	16
5	Matriks Tabel Basis Data Relasional (learnhire_db)	18
6	Panduan Pengoperasian & Akun Pengujian	19
6.1	Kredensial Akun Demonstrasi Bawaan	19
6.2	Prosedur Menjalankan Sistem Secara Mandiri	19

1. Pengantar & Peta Arsitektur Tiga Tingkat (Three-Tier)

Aplikasi **LearnHire** dirancang dengan pola arsitektur perangkat lunak *Three-Tier Architecture* yang bersih (*Clean Architecture*). Prinsip utama dari arsitektur ini adalah pemisahan tanggung jawab (*Separation of Concerns*), di mana antarmuka pengguna tidak boleh bersentuhan langsung dengan logika manipulasi data maupun koneksi basis data mentah.

1.1. Alur Interaksi Antar Layer

Setiap interaksi pengguna di layar melewati rute terstruktur sebagai berikut:

1. **Presentation Layer (Frontend):** Pengguna mengisi formulir atau mengklik aksi pada browser (HTML/CSS). Skrip JavaScript klien (`assets/js/pages/*.js`) menangkap aksi tersebut dan memanggil fungsi pembungkus `api.js`.
2. **Transport & Networking (Fetch API):** Permintaan dikirimkan melalui protokol HTTP/HTTPS menggunakan `fetch()` dengan payload JSON dan header `Authorization: Bearer <JWT>`.
3. **Routing & Middleware Layer (Express.js):** Server menerima request di `backend/server.js`, lalu diarahkan ke sub-rute spesifik (`backend/routes/*.routes.js`). Rute dilindungi oleh middleware penjaga (`auth.middleware.js`) yang memeriksa keabsahan token JWT dan status hak akses (*role*).
4. **Controller Layer (Business Logic):** Controller (`backend/controllers/*.controller.js`) memvalidasi parameter input, memproses aturan bisnis (misalnya hashing password atau pengecekan duplikasi lamaran), dan memanggil fungsi pada Model.
5. **Model Layer (Data Abstraction):** Model (`backend/models/*.model.js`) menyusun query SQL menggunakan parameter aman (*Prepared Statements*) untuk dieksekusi oleh pool koneksi MySQL.
6. **Database Layer (MySQL):** Basis data `learnhire_db` memproses query relasional, mengecek batasan (*Primary Key, Foreign Key, Unique Constraints*), dan mengembalikan baris data ke Model.
7. **Response Cycle:** Hasil dikembalikan secara berjenjang ke Controller, dibungkus dalam format JSON seragam, lalu dikirim kembali ke Frontend untuk memperbarui tampilan UI secara dinamis.

2. Bedah Berkas Root & Direktori Database

2.1. Berkas pada Root Direktori ([learnhire/](#))

`.gitignore`

[\[Konfigurasi Pengabaian Git\]](#)

Deskripsi & Fungsi: Menentukan berkas dan folder lokal yang tidak boleh dimasukkan ke dalam pelacakan repositori Git. Mencegah berkas-berkas sensitif dan berukuran besar terunggah ke publik.

Keterkaitan Berkas (Data Flow): Mengabaikan folder `node_modules/` (karena dependensi diinstal via `package.json`), berkas lingkungan `.env` (yang menyimpan password database dan rahasia JWT), berkas log eksekusi (`*.log`), dan berkas sementara sistem operasi (`.DS_Store`, `Thumbs.db`).

`README.md`

[\[Dokumentasi Pengguna & Manual Instalasi\]](#)

Deskripsi & Fungsi: Buku petunjuk komprehensif bagi pengembang dan penguji perangkat lunak. Memuat deskripsi LearnHire, ringkasan fitur, daftar teknologi, prasyarat instalasi (Node.js LTS, XAMPP), langkah konfigurasi basis data, panduan menjalankan server Express, daftar kredensial akun demonstrasi, spesifikasi ringkas REST API, dan solusi pemecahan masalah (*troubleshooting*).

Keterkaitan Berkas (Data Flow): Menjadi pintu masuk utama bagi siapa pun yang baru membuka repositori untuk memahami cara kerja dan cara menjalankan seluruh berkas di folder `backend/` dan `frontend/`.

2.2. Direktori Basis Data ([database/](#))

[\[DIREKTORI\]](#) `database/`

Peran Arsitektur: Folder penyimpanan skrip inisialisasi basis data relasional MySQL.

`database/database.sql`

[\[Skrip DDL & DML Skema MySQL\]](#)

Deskripsi & Fungsi: Skrip SQL komprehensif yang siap dieksekusi langsung melalui fitur *Import* phpMyAdmin atau terminal MySQL. Berisi instruksi pembuatan database `learnhire_db`, deklarasi 9 tabel normal dengan relasi kunci asing (*Foreign Key*), indeks performa, serta penyisipan data awal (*seeding*) sebanyak 7 pengguna (admin dan user reguler dengan hash Bcrypt valid), 6 perusahaan mitra ternama, 12 lowongan kerja, 10 kursus online terstruktur, riwayat lamaran, keikutsertaan kelas, dan tiket customer service.

Keterkaitan Berkas (Data Flow): Diimpor ke layanan MySQL (XAMPP). Seluruh kueri pada berkas-berkas di dalam folder `backend/models/` bergantung penuh pada keberadaan tabel dan kolom yang didefinisikan di berkas ini.

3. Bedah Direktori Backend (backend/)

Direktori `backend/` adalah inti pemrosesan logika server, otorisasi keamanan, dan pengelola jembatan komunikasi antara basis data MySQL dan antarmuka web.

3.1. Berkas Konfigurasi & Entry Point Root Backend

`backend/server.js`

[Entry Point Aplikasi Express.js]

Deskripsi & Fungsi: Berkas utama yang dijalankan saat perintah `npm run dev` atau `node server.js` dieksekusi. Berfungsi memuat pustaka Express, membaca konfigurasi `.env`, mengaktifkan middleware keamanan `helmet` (dengan pengaturan CSP permisif agar CDN font Poppins dan Font Awesome dapat dimuat), mengaktifkan `cors()` untuk request lintas-origin, dan `express.json()` untuk mengurai body request. Selain itu, berkas ini melayani file statis antarmuka dari direktori `../frontend`, me-mount master router di `/api`, menangani rute tak dikenal (*SPA fallback*), dan mendaftarkan middleware penanganan error.

Keterkaitan Berkas (Data Flow): Mengimpor middleware dari `backend/middleware/error.middleware.js` dan seluruh daftar rute dari `backend/routes/index.js`.

`backend/package.json`

[Manifes Dependensi Node.js / NPM]

Deskripsi & Fungsi: Mendefinisikan metadata proyek backend, skrip eksekusi ("`start`": "`node server.js`" dan "`dev`": "`nodemon server.js`"), serta daftar dependensi pustaka:

- `express`: Framework web HTTP server minimalis.
- `mysql2`: Driver komunikasi MySQL berkinerja tinggi dengan dukungan Promise.
- `bcryptjs`: Pustaka algoritma hashing kata sandi satu arah.
- `jsonwebtoken`: Pustaka pembuatan dan verifikasi token otentikasi JWT.
- `helmet`: Pustaka pengaman HTTP response headers.
- `cors`: Pengatur izin Cross-Origin Resource Sharing.
- `dotenv`: Pemuat variabel lingkungan dari berkas `.env`.
- `nodemon`: Utilitas auto-restart server saat terdeteksi perubahan kode.

Keterkaitan Berkas (Data Flow): Dipakai oleh manajer paket `npm` untuk mengunduh modul ke folder `node_modules/`.

`backend/.env` & `backend/.env.example`

[Konfigurasi Variabel Lingkungan]

Deskripsi & Fungsi: Berkas kunci rahasia yang menyimpan konfigurasi dinamis sistem: `PORT=3000`, `DB_HOST=localhost`, `DB_PORT=3306`, `DB_USER=root`, `DB_PASSWORD=`, `DB_NAME=learnhire_db`, `JWT_SECRET=learnhire_jwt_secret_key_2024_change_this`, dan `JWT_EXPIRES_IN=24h`. Berkas `.env.example` berfungsi sebagai cetak biru publik tanpa mengekspos kredensial produksi.

Keterkaitan Berkas (Data Flow): Dibaca secara global oleh `dotenv` pada baris pertama `backend/server.js`, `backend/config/database.js`, dan controller autentikasi.

3.2. Subdirektori Konfigurasi (backend/config/)

[DIREKTORI] backend/config/

Peran Arsitektur: Pusat konfigurasi infrastruktur dan layanan pihak ketiga.

backend/config/database.js

[Manajer Koneksi Basis Data (Connection Pool)]

Deskripsi & Fungsi: Menginisialisasi koneksi ke server MySQL menggunakan metode `mysql.createPool()`. Connection Pool mengizinkan Express menggunakan kembali koneksi yang sudah terbuka tanpa harus membuat koneksi TCP baru pada setiap request, yang secara drastis meningkatkan kecepatan respon dan mencegah server kehabisan memori. Berkas ini mengeksport `pool.promise()` sehingga seluruh model dapat menggunakan sintaks modern `async/await`.

Keterkaitan Berkas (Data Flow): Diimpor oleh setiap berkas model di dalam folder `backend/models/` untuk mengeksekusi perintah SQL.

3.3. Subdirektori Dokumentasi API (backend/docs/)

[DIREKTORI] backend/docs/

Peran Arsitektur: Dokumentasi teknis internal backend.

backend/docs/api.md

[Spesifikasi Rinci REST API Endpoint]

Deskripsi & Fungsi: Dokumen referensi teknis yang membedah setiap endpoint: HTTP Method, URL lengkap, header otorisasi, contoh JSON request body, contoh respon sukses (HTTP 200/201), format respon galat (HTTP 400/401/403/404/500), serta aturan batasan bisnis bagi pengembang antarmuka.

Keterkaitan Berkas (Data Flow): Dijadikan acuan dalam pengembangan fungsi-fungsi pada `frontend/assets/js/pages/*.js`.

3.4. Subdirektori Middleware (backend/middleware/)

[DIREKTORI] backend/middleware/

Peran Arsitektur: Fungsi penyaring request yang dieksekusi sebelum request mencapai controller tujuan.

backend/middleware/auth.middleware.js

[Penjaga Keamanan Token JWT & Peran Akun]

Deskripsi & Fungsi: Menyediakan tiga fungsi middleware krusial:

1. `verifyToken`: Mengambil token dari header `Authorization: Bearer <token>`. Jika token tidak ada atau tidak valid/kedaluwarsa, request langsung dipotong dengan respon HTTP 401 (*Unauthorized*). Jika valid, isi token didekode dan ditempelkan ke objek `req.user`.
2. `isAdmin`: Memeriksa apakah `req.user.role === 'admin'`. Jika bukan admin, server menolak akses dengan HTTP 403 (*Forbidden*).
3. `optionalAuth`: Mengekstrak identitas jika token disertakan, tetapi tidak memblokir request jika pengguna belum login (berguna untuk endpoint katalog).

Keterkaitan Berkas (Data Flow): Dipasang pada rute-rute di `backend/routes/*.routes.js` yang memerlukan hak akses khusus.

`backend/middleware/error.middleware.js`[\[Penanganan Error Terpusat\]](#)

Deskripsi & Fungsi: Menyediakan fungsi `notFound` (menangkap URL rute yang tidak terdaftar dan menghasilkan respon HTTP 404 JSON seragam) serta fungsi `errorHandler` (menangkap seluruh galat internal server HTTP 500 tanpa mengekspos rincian kueri database yang sensitif kepada pengguna umum).

Keterkaitan Berkas (Data Flow): Didaftarkan di baris akhir `backend/server.js` setelah seluruh rute di-mount.

3.5. Subdirektori Model Data (backend/models/)

[DIREKTORI] backend/models/

Peran Arsitektur: Layer abstraksi kueri SQL. Berisi representasi entitas data dan eksekusi prepared statements aman.

backend/models/user.model.js

[Model Entitas Pengguna (Users)]

Deskripsi & Fungsi: Menyediakan fungsi manipulasi tabel `users`:

- `findAll()`: Mengambil daftar seluruh user (tanpa kolom password).
- `findById(id)`: Mengambil profil user berdasarkan Primary Key ID.
- `findByEmail(email)`: Mengambil data user beserta hash password untuk verifikasi login.
- `create(userData)`: Menyimpan user baru dengan nama, email, password ter-hash, nomor telepon, dan role.
- `update(id, userData)`: Menggunakan kueri cerdas berbasis `COALESCE(?, nama_kolom)`, sehingga jika user hanya memperbarui bio atau nomor telepon, data keahlian dan alamat sebelumnya tidak akan terhapus atau tertimpa menjadi NULL.
- `delete(id)`: Menghapus akun user dari sistem.

Keterkaitan Berkas (Data Flow): Menggunakan koneksi dari `config/database.js`. Dipanggil oleh `controllers/auth.controller.js`.

backend/models/job.model.js

[Model Entitas Lowongan Kerja (Jobs)]

Deskripsi & Fungsi: Mengelola kueri tabel `jobs`. Fungsi `findAll(filters)` melakukan operasi JOIN ke tabel `companies` dan `job_categories` untuk menyajikan nama perusahaan, logo perusahaan, dan nama kategori secara utuh. Fungsi ini mendukung filter dinamis berdasarkan kata kunci pencarian (LIKE), kategori (`category_id`), tipe kerja (Full-time/Part-time/Contract/Internship), dan status aktif. Juga menyediakan fungsi `findById`, `create`, `update`, `delete`, dan `countAll`.

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/job.controller.js` dan `controllers/dashboard.controller.js`.

backend/models/company.model.js

[Model Entitas Perusahaan Mitra (Companies)]

Deskripsi & Fungsi: Mengelola data master instansi mitra pada tabel `companies`: nama perusahaan, logo, deskripsi profil, lokasi kantor, alamat website, dan sektor industri. Menyediakan fungsi CRUD lengkap (`findAll`, `findById`, `create`, `update`, `delete`).

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/company.controller.js`. Menjadi target relasi Foreign Key bagi model lowongan kerja.

backend/models/course.model.js

[Model Entitas Kursus Pelatihan (Courses)]

Deskripsi & Fungsi: Mengelola katalog kursus pada tabel `courses`. Fungsi `findAll(filters)` melakukan JOIN ke tabel `course_categories` dan mendukung penyaringan berdasarkan kata kunci judul, instruktur, ID kategori, dan tingkat keahlian (`level`: *Beginner*, *Intermediate*, *Advanced*). Juga mengelola informasi durasi, harga kelas, rating, serta jumlah total siswa yang terdaftar.

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/course.controller.js` dan diinkremen otomatis oleh model `enrollment`.

`backend/models/application.model.js`[\[Model Transaksi Lamaran Pekerjaan \(Job Applications\)\]](#)

Deskripsi & Fungsi: Mengelola tabel relasional `job_applications`:

- `findById(userId)`: Mengambil daftar lamaran yang diajukan oleh user tertentu beserta detail judul pekerjaan dan nama perusahaannya.
- `findAll()`: Mengambil seluruh berkas lamaran dari semua pengguna (khusus untuk hak akses administrator).
- `checkExisting(userId, jobId)`: Memeriksa apakah user sudah pernah melamar di posisi tersebut untuk mencegah pengiriman berkas ganda.
- `create()`, `update()`, `delete()`: Menyimpan pengajuan lamaran, mengubah status seleksi (*pending*, *reviewed*, *accepted*, *rejected*), dan membatalkan lamaran.

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/application.controller.js`. Menghubungkan tabel `users` dan `jobs`.

`backend/models/enrollment.model.js`[\[Model Pendaftaran Kursus \(Course Enrollments\)\]](#)

Deskripsi & Fungsi: Mengelola tabel `course_enrollments`. Fungsi `create()` menyimpan pendaftaran kelas baru sekaligus mengeksekusi kueri otomatis untuk menambah counter `total_students` pada tabel `courses`. Fungsi `update()` memperbarui persentase pemahaman materi (0% hingga 100%) dan otomatis membubuhkan timestamp `completed_at` saat kursus dituntaskan. Fungsi `delete()` membatalkan kelas dan mengurangi counter siswa.

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/enrollment.controller.js`. Menghubungkan tabel `users` dan `courses`.

`backend/models/category.model.js`[\[Model Kategori Referensi \(Categories\)\]](#)

Deskripsi & Fungsi: Menyediakan fungsi pembacaan data master kategori: `findAllJobCategories()` untuk kategori pekerjaan (Teknologi, Desain, Marketing, Finance, Engineering, Data Science) dan `findAllCourseCategories()` untuk bidang kursus (Programming, UI/UX Design, Data Science, Digital Marketing, Business, Language).

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/category.controller.js` untuk mengisi elemen `<select>` dropdown pada antarmuka pencarian frontend.

`backend/models/support.model.js`[\[Model Layanan Bantuan \(Support Messages\)\]](#)

Deskripsi & Fungsi: Mengelola tiket komunikasi pelanggan pada tabel `support_messages`. Menyediakan fungsi `create` (pengajuan tiket baru oleh user), `findByUserId` (membaca riwayat kendala pribadi user), `findAll` (menampilkan seluruh tiket komplain untuk admin), `update` (admin memberikan balasan jawaban dan mengubah status tiket menjadi *resolved*), serta `delete`.

Keterkaitan Berkas (Data Flow): Dipanggil oleh `controllers/support.controller.js`. Menghubungkan pelapor ke entitas user.

3.6. Subdirektori Controller Backend (`backend/controllers/`)

[\[DIREKTORI\] `backend/controllers/`](#)

Peran Arsitektur: Pusat kendali logika bisnis aplikasi yang memvalidasi input HTTP dan mengembalikan JSON.

`backend/controllers/auth.controller.js`[\[Pengendali Autentikasi & Profil Akun\]](#)

Deskripsi & Fungsi: Mengelola 4 operasi vital:

1. **register:** Memvalidasi keberadaan nama, email, nomor telepon, dan password. Memeriksa keunikan email via model user. Mengenkripsi password dengan salt rounds 10 via `bcryptjs`. Menyimpan akun baru dan langsung menghasilkan token JWT aktif berumur 24 jam.
2. **login:** Memeriksa kecocokan email dan mencocokkan kata sandi menggunakan `bcrypt.compare()`. Jika cocok, membuat token JWT yang memuat identitas { `id`, `name`, `role` } untuk dikembalikan ke browser klien.
3. **getProfile:** Membaca data lengkap akun pengguna yang sedang login berdasarkan `req.user.id`.
4. **updateProfile:** Menyimpan pembaruan data diri (nama, telepon, keahlian, bio) ke basis data.

Keterkaitan Berkas (Data Flow): Menggunakan `models/user.model.js`. Dipanggil oleh rute di `routes/auth.routes.js`.

`backend/controllers/job.controller.js`[\[Pengendali Katalog Lowongan Kerja\]](#)

Deskripsi & Fungsi: Menyediakan method `getAll` (mengambil lowongan dengan filter query), `getById` (mengambil detail lengkap satu lowongan), `create` (menambah lowongan baru dengan validasi role admin), `update` (mengubah kualifikasi/gaji lowongan), dan `delete` (menghapus lowongan).

Keterkaitan Berkas (Data Flow): Menggunakan `models/job.model.js`. Dipanggil oleh rute di `routes/job.routes.js`.

`backend/controllers/company.controller.js`[\[Pengendali Profil Perusahaan Mitra\]](#)

Deskripsi & Fungsi: Mengelola operasi CRUD data perusahaan mitra (`getAll`, `getById`, `create`, `update`, `delete`). Operasi modifikasi data diproteksi ketat hanya untuk admin.

Keterkaitan Berkas (Data Flow): Menggunakan `models/company.model.js`. Dipanggil oleh rute di `routes/company.routes.js`.

`backend/controllers/course.controller.js`[\[Pengendali Katalog Kursus Online\]](#)

Deskripsi & Fungsi: Menyediakan method `getAll` (mengambil seluruh kursus aktif beserta filter tingkat dan kategori), `getById` (mengambil detail kursus dan silabus), `create` (penambahan materi kursus baru oleh admin), `update` (pembaruan harga, deskripsi, atau durasi), dan `delete`.

Keterkaitan Berkas (Data Flow): Menggunakan `models/course.model.js`. Dipanggil oleh rute di `routes/course.routes.js`.

`backend/controllers/application.controller.js`[\[Pengendali Alur Pelamaran Kerja\]](#)

Deskripsi & Fungsi: Mengimplementasikan logika bisnis pelamaran kerja:

- `getAll`: Secara cerdas memeriksa role user. Jika `role === 'admin'`, mengembalikan seluruh lamaran masuk dari seluruh pelamar. Jika role adalah user biasa, hanya mengembalikan berkas lamaran milik akun yang sedang login.
- `create`: Mencegah pelamar melamar dua kali pada lowongan yang sama menggunakan `checkExisting`. Menyimpan surat lamaran (*cover letter*) dan URL resume.
- `update`: Mengubah status lamaran (*pending, reviewed, accepted, rejected*) yang dikhususkan bagi admin setelah menerima konfirmasi dari HRD mitra.
- `delete`: Membatalkan pengajuan lamaran dengan memastikan kepemilikan akun.

Keterkaitan Berkas (Data Flow): Menggunakan `models/application.model.js`. Dipanggil oleh rute di `routes/application.routes.js`.

`backend/controllers/enrollment.controller.js`[\[Pengendali Pendaftaran & Progres Kursus\]](#)

Deskripsi & Fungsi: Mengelola proses pendaftaran kelas pengguna (`create`) dengan validasi agar user tidak mendaftar dua kali di kelas yang sama. Menyediakan fungsi `getAll` untuk melihat kelas yang sedang dipelajari, fungsi `update` untuk memperbarui persentase progres belajar dari antarmuka, serta fungsi `delete` untuk membatalkan kelas.

Keterkaitan Berkas (Data Flow): Menggunakan `models/enrollment.model.js`. Dipanggil oleh rute di `routes/enrollment.routes.js`.

`backend/controllers/category.controller.js`[\[Pengendali Kategori Sistem\]](#)

Deskripsi & Fungsi: Menyediakan method `getJobCategories` dan `getCourseCategories` yang mengembalikan daftar kategori dalam format array JSON siap pakai untuk merender opsi filter pada antarmuka klien.

Keterkaitan Berkas (Data Flow): Menggunakan `models/category.model.js`. Dipanggil oleh rute di `routes/category.routes.js`.

`backend/controllers/support.controller.js`[\[Pengendali Tiket Bantuan Pelanggan\]](#)

Deskripsi & Fungsi: Mengelola komunikasi layanan pelanggan: `getAll` (mengambil tiket kendala milik user atau seluruh tiket jika admin), `create` (pengiriman tiket baru), `update` (admin menuliskan respon pemecahan masalah dan menutup status tiket), serta `delete`.

Keterkaitan Berkas (Data Flow): Menggunakan `models/support.model.js`. Dipanggil oleh rute di `routes/support.routes.js`.

`backend/controllers/dashboard.controller.js`[\[Pengendali Statistik Ringkasan Sistem\]](#)

Deskripsi & Fungsi: Fungsi `getStats` mengeksekusi kueri agregasi `COUNT(*)` secara simultan dari tabel `jobs`, `courses`, `users`, dan `job_applications`. Mengembalikan total angka statistik riil untuk ditampilkan pada komponen kartu metrik beranda web.

Keterkaitan Berkas (Data Flow): Mengakses koneksi database langsung secara ringkas. Dipanggil oleh `routes/dashboard.routes.js`.

3.7. Subdirektori Routing Backend (backend/routes/)

[DIREKTORI] backend/routes/

Peran Arsitektur: Mendefinisikan pemetaan endpoint URL ke controller serta memasang middleware otentikasi.

backend/routes/index.js

[Master Router Aggregator]

Deskripsi & Fungsi: Menginstansiasi `express.Router()` utama dan me-mount seluruh modul rute anak ke bawah path spesifik: `/auth` → `auth.routes`, `/jobs` → `job.routes`, `/companies` → `company.routes`, `/courses` → `course.routes`, `/applications` → `application.routes`, `/enrollments` → `enrollment.routes`, `/categories` → `category.routes`, `/support` → `support.routes`, dan `/dashboard` → `dashboard.routes`.

Keterkaitan Berkas (Data Flow): Diimpor langsung oleh `backend/server.js` pada baris `app.use('/api', require('./routes'))`.

backend/routes/*.routes.js (9 Berkas Sub-Rute)

[Pemetaan Rute HTTP Per Modul]

Deskripsi & Fungsi: Setiap berkas menghubungkan HTTP Verb spesifik ke fungsi controller yang bersesuaian serta menyematkan middleware pelindung:

- `auth.routes.js`: `POST /register`, `POST /login`, `GET /profile` (*verifyToken*), `PUT /profile` (*verifyToken*).
- `job.routes.js`: `GET /`, `GET /:id`, `POST /` (*verifyToken*, *isAdmin*), `PUT /:id` (*verifyToken*, *isAdmin*), `DELETE /:id` (*verifyToken*, *isAdmin*).
- `company.routes.js`: `GET /`, `GET /:id`, `POST /`, `PUT /:id`, `DELETE /:id` (*isAdmin*).
- `course.routes.js`: `GET /`, `GET /:id`, `POST /`, `PUT /:id`, `DELETE /:id` (*isAdmin*).
- `application.routes.js`: `GET /`, `POST /`, `PUT /:id`, `DELETE /:id` (semua dilindungi *verifyToken*).
- `enrollment.routes.js`: `GET /`, `POST /`, `PUT /:id`, `DELETE /:id` (semua dilindungi *verifyToken*).
- `category.routes.js`: `GET /jobs`, `GET /courses` (akses publik).
- `support.routes.js`: `GET /`, `POST /`, `PUT /:id`, `DELETE /:id` (dilindungi *verifyToken*).
- `dashboard.routes.js`: `GET /` (akses publik statistik sistem).

Keterkaitan Berkas (Data Flow): Diimpor dan digabungkan oleh berkas `backend/routes/index.js`.

4. Bedah Direktori Frontend (frontend/)

Direktori `frontend/` menyusun seluruh antarmuka pengguna (*User Interface*) berbasis teknologi standar web modern murni: HTML5 semantik, CSS3 Glassmorphism, dan Vanilla JavaScript ES6+.

4.1. Landing Page Publik & Komponen Bersama

`frontend/index.html`

[Landing Page Utama (Canva Begin Page)]

Deskripsi & Fungsi: Halaman penyambutan publik yang merepresentasikan layar *Begin Page* pada prototipe Canva. Memuat latar belakang gelap dengan animasi lingkaran cahaya melayang (*floating orbs*), judul brand bergradien ungu-ke-biru (*LearnHire*), tagline inspiratif, tombol ajakan "*Mulai Sekarang*" yang mengarahkan ke formulir masuk, kartu sorotan 3 fitur utama (Lowongan Kerja, Kursus Online, dan CV Builder), counter metrik statistik dinamis dari API, serta footer hak cipta. Halaman ini juga memiliki skrip pemeriksaan status: jika user telah memiliki sesi login aktif, otomatis dialihkan langsung ke dashboard `home.html`.

Keterkaitan Berkas (Data Flow): Memuat berkas `assets/css/style.css` dan pustaka klien `api.js`, `auth.js`, serta `app.js`.

`frontend/components/sidebar.html`

[Komponen Navigasi Samping Responsif]

Deskripsi & Fungsi: Komponen bilah navigasi yang diinjeksi secara dinamis ke seluruh halaman privat. Memuat logo brand LearnHire, daftar navigasi utama berikan Font Awesome (Beranda, Lowongan Kerja, Kursus Online, Lamaran Saya, Kursus Saya, Profil, Bantuan), serta bagian footer profil yang menampilkan avatar inisial, nama pengguna, rencana status peran ([ADMIN] warna oranye atau [USER] warna biru), dan tombol keluar (*Logout*).

Keterkaitan Berkas (Data Flow): Dimuat secara asinkron oleh fungsi `app.loadSidebar()` yang berada di `assets/js/app.js`.

4.2. Pusat Gaya Antarmuka (frontend/assets/css/)

[DIREKTORI] `frontend/assets/css/`

Peran Arsitektur: Pengelolaan tata letak visual, tema warna, dan responsivitas.

`frontend/assets/css/style.css`

[Pusat Desain Dark Glassmorphism]

Deskripsi & Fungsi: Berkas CSS menyeluruh yang mengimplementasikan identitas visual LearnHire:

- **Palette Warna:** Latar belakang utama (`-primary-bg: #0a0e27`), sekunder (`-secondary-bg: #1a1f4e`), kartu kaca transparan (`-card-bg: rgba(255,255,255,0.05)`), aksent gradien (`#7c3aed` ke `#3b82f6`), teks utama putih, dan teks sekunder abu-abu (`#94a3b8`).
- **Efek Kaca (Glassmorphism):** Menggunakan properti `backdrop-filter: blur(10px)` dan border semi-transparan untuk menciptakan kedalaman visual modern.
- **Elemen Orbs Melayang:** Mendefinisikan animasi CSS `@keyframes float` yang menggerakkan lingkaran blur berwarna ungu dan biru di latar belakang secara halus.
- **Sistem Grid & Responsivitas:** Mendefinisikan class `.grid-2`, `.grid-3`, dan `.stats-grid`. Pada layar desktop (`> 1024px`), sidebar berukuran tetap 260px; pada tablet (`≤ 1024px`), sidebar mengecil secara elegan; dan pada ponsel (`≤ 768px`), sidebar tersembunyi dengan tombol pemicu hamburger menu di bar atas.
- **Komponen UI Mandiri:** Tombol gradien, input form dengan ikon dan glow fokus, status badge warna-warni, jendela modal, toast notifikasi animasi slide-in, dan indikator loading spinner.

Keterkaitan Berkas (Data Flow): Diimpor di bagian `<head>` oleh seluruh 11 halaman HTML di dalam proyek LearnHire.

4.3. Pustaka JavaScript Klien Bersama (frontend/assets/js/)

[DIREKTORI] frontend/assets/js/

Peran Arsitektur: Modul-modul utilitas logika klien yang digunakan lintas halaman.

frontend/assets/js/api.js

[Pustaka Komunikasi Fetch API Terpusat]

Deskripsi & Fungsi: Menyediakan objek global `api` dengan method pembungkus HTTP: `api.get()`, `api.post()`, `api.put()`, dan `api.delete()`. Berkas ini membaca token otentikasi dari `auth.getToken()` dan otomatis membubuhkannya pada header `Authorization: Bearer <token>`. Jika server mengembalikan HTTP 401 (*Unauthorized*), berkas ini otomatis menghapus sesi lokal dan mengalihkan halaman ke formulir login.

Keterkaitan Berkas (Data Flow): Menjadi jalur komunikasi tunggal bagi seluruh berkas skrip di folder `assets/js/pages/`.

frontend/assets/js/auth.js

[Manajer Sesi Autentikasi Pengguna]

Deskripsi & Fungsi: Mengelola penyimpanan token JWT di `localStorage`. Menyediakan fungsi:

- `getToken() & setToken(token)`: Mengakses dan menyimpan string token JWT.
- `isLoggedIn()`: Mengembalikan nilai boolean apakah sesi aktif tersedia.
- `checkAuth()`: Pelindung halaman privat; jika belum login, otomatis me-redirect ke `login.html`.
- `checkGuest()`: Pelindung halaman login/register; jika sudah login, otomatis me-redirect ke `home.html`.
- `logout()`: Menghapus token dari memori browser dan mengarahkan kembali ke landing page `index.html`.
- `parseJwt(token) & getUser()`: Mendekode bagian payload JWT secara aman untuk membaca nama dan peran pengguna di sisi klien tanpa perlu memanggil server.

Keterkaitan Berkas (Data Flow): Dipanggil pada baris awal setiap skrip halaman privat untuk menjamin otentikasi pengguna.

frontend/assets/js/app.js

[Pustaka Utilitas Antarmuka Global]

Deskripsi & Fungsi: Menyediakan fungsi pendukung pengalaman pengguna:

- `showNotification(message, type)`: Menghasilkan pesan toast melayang animasi di pojok layar dengan opsi warna (sukses, galat, peringatan, info).
- `formatCurrency(amount)`: Mengonversi angka ke format Rupiah baku (contoh: Rp 10.000.000).
- `formatDate(dateString)`: Mengonversi format ISO timestamp ke penanggalan berbahasa Indonesia.
- `loadSidebar()`: Memuat komponen `sidebar.html`, menandai link aktif sesuai halaman saat ini, dan memasang event listener hamburger menu.
- `updateUserInfo()`: Menampilkan nama user di sidebar, membuat inisial avatar, serta menampilkan rencana peran secara akurat ([ADMIN] oranye atau [USER] biru).
- `showLoading() & showEmptyState()`: Menampilkan animasi spinner saat data sedang diambil dan tampilan state kosong jika tidak ada data yang ditemukan.

Keterkaitan Berkas (Data Flow): Dijalankan pada saat event `DOMContentLoaded` aktif di setiap halaman aplikasi.

4.4. Halaman Web Modular (frontend/pages/)

[DIREKTORI] frontend/pages/

Peran Arsitektur: Kumpulan berkas tampilan HTML untuk setiap fungsionalitas sistem.

1. **login.html:** Halaman formulir masuk terpusat. Digunakan bersama baik oleh Administrator maupun Pelamar Kerja. Menyediakan input email dan kata sandi dengan ikon penjelas.
2. **register.html:** Halaman pendaftaran akun pencari kerja baru. Memuat kolom input Nama Lengkap, Alamat Email, Nomor Telepon, Kata Sandi, dan Konfirmasi Kata Sandi.
3. **home.html:** Halaman beranda utama pengguna terautentikasi. Menyajikan salam personal (*"Selamat Datang, [Nama]!"*), 4 kartu metrik statistik (Total Jobs, Total Courses, Lamaran Saya, Kursus Saya), serta carousel cuplikan lowongan pekerjaan dan kursus pelatihan terpopuler.
4. **jobs.html:** Halaman eksplorasi lowongan pekerjaan. Dilengkapi kartu filter pencarian posisi dan dropdown pemilihan kategori industri yang terisi secara dinamis dari API.
5. **job-detail.html:** Halaman detail lowongan pekerjaan tertentu. Menyajikan logo dan profil perusahaan mitra, kisaran gaji, lokasi kerja, deskripsi pekerjaan, daftar persyaratan, tombol kirim berkas, serta jendela modal formulir surat lamaran (*cover letter*).
6. **courses.html:** Halaman katalog pelatihan kursus online. Menyajikan kartu-kartu materi keahlian lengkap dengan foto modul, instruktur pengajar, tingkat kesulitan (*Beginner, Intermediate, Advanced*), durasi pembelajaran, rating kepuasan bintang, dan jumlah peserta terdaftar.
7. **course-detail.html:** Halaman rincian silabus kursus. Menyediakan penjelasan kompetensi yang akan dicapai, rincian bab silabus belajar, tombol konfirmasi pendaftaran kelas, serta otomatis menampilkan bilah progres belajar jika pengguna telah terdaftar di kelas tersebut.
8. **applications.html:** Halaman pemantauan riwayat lamaran kerja pengguna. Menampilkan tabel berisi posisi pekerjaan, nama instansi mitra, tanggal pengajuan, status seleksi berwarna (*Pending, Reviewed, Accepted, Rejected*), serta tombol pembatalan berkas lamaran.
9. **my-courses.html:** Halaman kursus aktif yang sedang diikuti pengguna. Dilengkapi bilah progres persentase belajar (*progress bar*) dan tombol interaktif "*Lanjutkan Belajar (+25%)*" yang langsung memperbarui capaian belajar ke basis data MySQL hingga tuntas 100%.
10. **profile.html:** Halaman pengelolaan profil diri. Menampilkan kartu identitas visual dengan rencana peran (*Administrator* atau *Pencari Kerja*), serta formulir pengubahan nama lengkap, nomor telepon, daftar keahlian kompetensi, dan bio ringkas.
11. **support.html:** Halaman pusat layanan bantuan pengguna (Customer Service). Menampilkan riwayat tiket pertanyaan kendala, status penanganan tiket (*Terbuka, Sedang Diproses, Selesai Dijawab*), balasan solusi dari tim admin, serta tombol pemicu modal pengajuan tiket bantuan baru.

4.5. Skrip Klien Per Halaman (frontend/assets/js/pages/)

[DIREKTORI] frontend/assets/js/pages/

Peran Arsitektur: Skrip JavaScript modular yang menangani event, pemanggilan API, dan manipulasi DOM per halaman.

Setiap berkas JavaScript pada folder ini terhubung secara eksklusif ke halaman HTML yang bersesuaian:

- **login.js:** Mencegah form reload (`e.preventDefault()`), memvalidasi kelengkapan isian, memanggil endpoint `api.post('/auth/login', ...)`, menyimpan token ke `localStorage` via `auth.setToken()`, memunculkan toast sukses, dan mengarahkan pengguna ke `home.html`.
- **register.js:** Memvalidasi kesamaan kata sandi dan konfirmasi kata sandi, memeriksa panjang karakter minimum, memanggil endpoint `api.post('/auth/register', ...)`, memunculkan notifikasi keberhasilan, dan mengarahkan ke halaman login.
- **home.js:** Memanggil endpoint agregat `/dashboard` untuk statistik global, endpoint `/applications` dan `/enrollments` untuk menghitung total lamaran dan kursus pribadi user, serta mengambil cuplikan 3 loker dan kursus teratas via `/jobs` dan `/courses`.
- **jobs.js:** Memanggil endpoint kategori `/categories/jobs` untuk merender dropdown opsi, lalu mengeksekusi pemanggilan `/jobs?search=...&category_id=...` berdasarkan filter yang diinputkan pengguna.
- **job-detail.js:** Membaca parameter ID dari query string URL (`?id=X`), memanggil `/jobs/:id`, memeriksa apakah lowongan sudah pernah dilamar oleh pengguna aktif, mengatur pembukaan modal lamaran, dan mengeksekusi pengiriman berkas ke endpoint `POST /applications`.
- **courses.js:** Mengambil daftar kategori kursus via `/categories/courses` dan merender kartu-kartu materi kursus dari endpoint `/courses` dengan filter dinamis.
- **course-detail.js:** Mengambil data rincian kelas via `/courses/:id`, memeriksa kepemilikan kelas via `/enrollments`, mengatur jendela modal konfirmasi pendaftaran, dan memicu pendaftaran kelas ke `POST /enrollments`.
- **applications.js:** Mengambil daftar lamaran aktif pengguna dari endpoint `GET /applications`, merender rencana status warna (*badge-pending*, *badge-info*, *badge-active*, *badge-rejected*), serta menangani tombol pembatalan lamaran yang memanggil endpoint `DELETE /applications/:id`.
- **my-courses.js:** Membaca kelas aktif dari `GET /enrollments`, menghitung lebar bilah progres CSS, dan menangani event klik tombol progres yang mengeksekusi kueri `PUT /enrollments/:id` dengan kenaikan progres +25% hingga status tuntas *completed*.
- **profile.js:** Mengambil data profil mutakhir pengguna dari endpoint `GET /auth/profile`, mengisi nilai formulir, mengatur rencana status akun, serta mengeksekusi pembaruan profil ke endpoint `PUT /auth/profile`.
- **support.js:** Mengambil riwayat tiket bantuan dari endpoint `GET /support`, menampilkan tanggapan resmi dari tim admin, mengatur pembukaan modal pembuatan tiket baru (`POST /support`), serta menangani penghapusan tiket (`DELETE /support/:id`).

5. Matriks Tabel Basis Data Relasional (learnhire_db)

Berikut adalah rincian 9 tabel relasional yang menyusun basis data aplikasi LearnHire:

Nama Tabel	Kolom-Kolom Kunci	Integritas Relasi	Deskripsi Fungsional
users	id, name, email, password, phone, role	PRIMARY KEY (id), UNIQUE (email)	Menyimpan akun pengguna dan administrator dengan kata sandi ter-hash Bcrypt.
companies	id, name, logo, location, website	PRIMARY KEY (id)	Profil institusi atau perusahaan mitra penyedia lowongan pekerjaan resmi.
job_categories	id, name, icon	PRIMARY KEY (id)	Klasifikasi bidang industri pekerjaan (Teknologi, Desain, Keuangan, Marketing).
jobs	id, company_id, category_id, title, salary_min, type	PRIMARY KEY (id), FK(company_id), FK(category_id)	Katalog lowongan pekerjaan aktif milik perusahaan mitra.
job_applications	id, user_id, job_id, cover_letter, status	PRIMARY KEY (id), FK(user_id), FK(job_id), UNIQUE(user_id, job_id)	Transaksi pelamaran kerja. Kunci unik mencegah duplikasi pengajuan berkas.
course_categories	id, name, icon	PRIMARY KEY (id)	Bidang keilmuan kursus (Programming, UI/UX, Data Science, Bisnis).
courses	id, category_id, title, instructor, duration, price	PRIMARY KEY (id), FK(category_id)	Katalog kursus pelatihan keahlian yang dapat diikuti secara terstruktur.
course_enrollments	id, user_id, course_id, progress, status	PRIMARY KEY (id), FK(user_id), FK(course_id), UNIQUE(user_id, course_id)	Pelacakan kelas yang diikuti pengguna beserta persentase kelulusan (0-100%).
support_messages	id, user_id, subject, message, reply, status	PRIMARY KEY (id), FK(user_id)	Komunikasi bantuan teknis dua arah antara pengguna dan administrator.

Table 1: Struktur Relasional 9 Tabel Basis Data learnhire_db

6. Panduan Pengoperasian & Akun Pengujian

6.1. Kredensial Akun Demonstrasi Bawaan

Untuk memfasilitasi pengujian alur seleksi dan perbedaan hak akses antara Admin dan User reguler, gunakan kredensial bawaan berikut:

Peran Akun	Alamat Email	Kata Sandi	Hak Akses & Batasan Fungsional
Administrator	admin@learnhire.com	admin123	Hak akses superuser: dapat melihat seluruh lamaran masuk dari semua pelamar, mengelola CUD lowongan dan kursus, serta membalas tiket bantuan.
Pencari Kerja	budi@example.com	user123	Pengguna reguler: telah memiliki riwayat pengajuan lamaran pekerjaan dan keikutsertaan kelas aktif untuk demonstrasi alur belajar.
Pencari Kerja	sari@example.com	user123	Akun alternatif: untuk menguji simulasi pendaftaran kursus baru dan pengajuan tiket bantuan teknis mandiri.

Table 2: Daftar Akun Pengujian Bawaan Sistem

6.2. Prosedur Menjalankan Sistem Secara Mandiri

- Menjalankan Layanan Basis Data (XAMPP):** Buka XAMPP Control Panel, pastikan modul **Apache** (port 80) dan **MySQL** (port 3306) telah berada pada status *Running*.
- Verifikasi phpMyAdmin:** Buka peramban web pada alamat <http://localhost/phpmyadmin/>. Pastikan database `learnhire_db` telah terbuat beserta 9 tabel relasionalnya. Jika belum, impor berkas `database/database.sql`.
- Menjalankan Server Backend Express:** Buka terminal (PowerShell atau Command Prompt) pada direktori backend:

```
cd "c:\Users\M S I\OneDrive - Institut Teknologi Sepuluh Nopember\Documents\ANTIGRAVITY\learnhire\backend"
npm run dev
```

Server akan aktif dan siap menerima panggilan API pada alamat <http://localhost:3000/api>.

- Mengakses Antarmuka Web:** Akses tautan <http://localhost:3000> pada browser untuk masuk ke Landing Page, atau buka <http://localhost:3000/pages/login.html> untuk login menggunakan akun demonstrasi di atas.

Dokumen ini disusun secara terperinci untuk mendokumentasikan setiap berkas dan direktori yang ada pada project LearnHire secara transparan dan terverifikasi.